

Ubiquitous Sensing, Computing & Communication Laboratory

Quality Laboratory Manual

Experiment No. 02

**To Understanding and Implementing LED ON-OFF Control Using
Raspberry Pi 4 Model B.**

**Course Instructor – Mr.
Samish N Kamble
ASSISTANT PROFESSOR**

Experiment No. 02

Title of Experiment: To Understanding and Implementing LED ON-OFF Control Using Raspberry Pi 4 Model B.

Aim of Experiment: To interface and control a single LED using Raspberry Pi 4 Model B and Python programming.

System Requirements – Raspberry pi 4 Board Win 10 and above OS, 4GB RAM, 2.33 GHz Processor.

Hardware:

- Raspberry Pi 4 Model B
- MicroSD card (with Raspberry Pi OS installed)
- LED (5mm, any color)
- 330Ω resistor
- Breadboard
- Jumper wires
- Power supply (5V, 3A USB-C adapter)

Software:

- Raspberry Pi OS
- Python 3
- RPi. GPIO library

Experiment Objectives:

- To understand GPIO pin configuration of Raspberry Pi 4.
- To interface and control an LED using Python programming.
- To learn the working of GPIO pins in output mode.
- To execute a Python script to turn an LED ON and OFF.

Experiment Outcomes:

- The LED will turn ON and OFF based on the program execution.
- The student will gain practical experience in **GPIO pin handling**.
- The student will understand the basics of **Python-based hardware programming**.
- The student will be able to apply this knowledge to control other hardware components like motors, sensors, and displays.

1.Theory:

1.GPIO (General Purpose Input/Output) Pins

- The Raspberry Pi 4 Model B has **40 GPIO pins** that can be used for input and output operations.
- Some pins provide **3.3V or 5V power**, while others are used for **data communication** (I2C, SPI, UART).
- Each GPIO pin can be configured as **INPUT** or **OUTPUT** based on requirements.

2. LED Control Using Raspberry Pi

- An LED (Light Emitting Diode) requires a current-limiting resistor (330Ω) to prevent damage.
- The LED is connected to a **GPIO pin** and **GND**.
- Setting the GPIO pin **HIGH (1)** turns the LED ON, and setting it **LOW (0)** turns it OFF.

3. Connections

Raspberry Pi Pin	Component
GPIO 17 (Pin 11)	LED Anode (+)
	LED Cathode (-) via 330Ω resistor
GND (Pin 6)	

4. Setting up the circuit

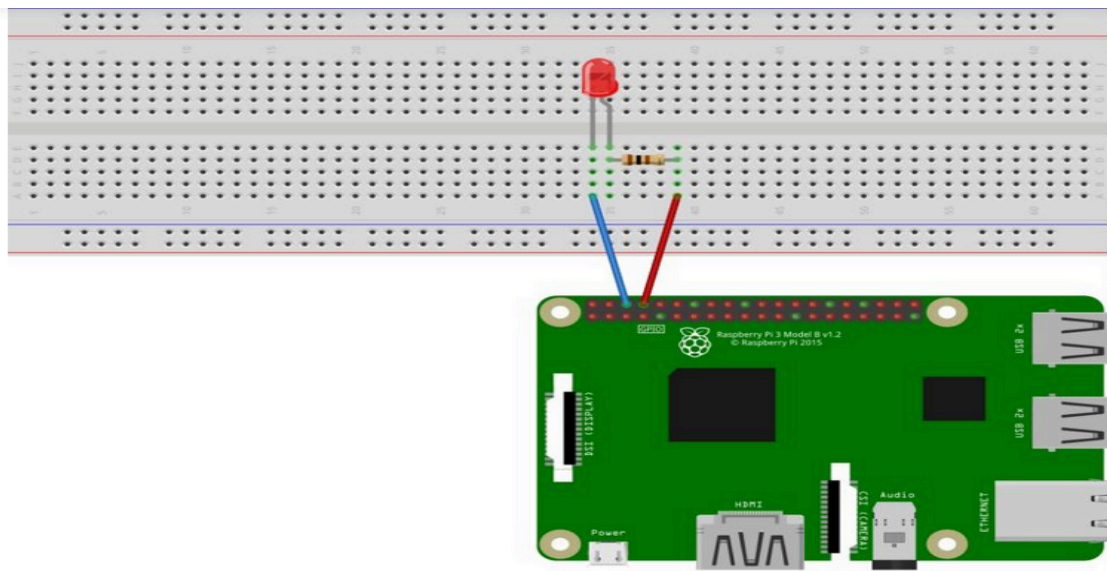
The first step in this project is to design a simple LED circuit. Then we will make the LED circuit controllable from the Raspberry Pi by connecting the circuit to the general purpose input/output (GPIO) pins on the Raspberry Pi.

A simple LED circuit consists of a LED and resistor. The resistor is used to limit the current that is being drawn and is called a *current limiting resistor*. Without the resistor the LED would run at too high of a voltage, resulting in too much current being drawn which in turn would instantly burn the LED, and likely also the GPIO port on the Raspberry Pi.

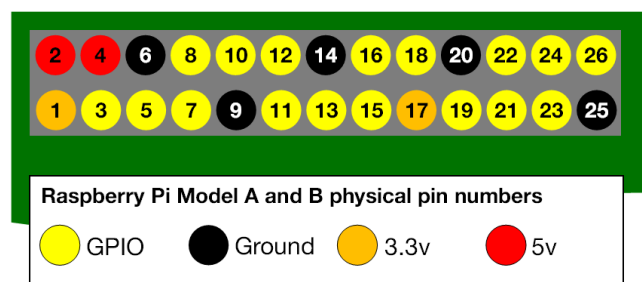
To calculate the resistor value we need to examine the specifications of the LED. Specifically we need to find the forward voltage (VF) and the forward current (IF). A regular red LED has a forward voltage (VF) of 1.7V and forward current of 20mA (IF). Additionally we need to know the output voltage of the Raspberry Pi which is 3.3V.

We can then calculate the resistor size needed to limit the current to the LED's maximum forward current (IF) using ohm's law like this:

Circuit Diagram:-



To find the pin number refer to this diagram showing the physical pin numbers on the Raspberry Pi.



5. Python Program to Turn LED ON and OFF

```
import RPi.GPIO as GPIO
import time

# Pin Configuration
LED_PIN = 17                                # GPIO17 (Pin 11)

# GPIO setup
GPIO.setmode(GPIO.BCM)                      # Use BCM numbering
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)     # Turn LED ON
        print("LED ON")
        time.sleep(1)                       # Wait 1 second

        GPIO.output(LED_PIN, GPIO.LOW)      # Turn LED OFF
        print("LED OFF")
        time.sleep(1)                       # Wait 1 second

except KeyboardInterrupt:
    print("Program Stopped")
    GPIO.cleanup()                          # Reset GPIO settings
```

6.Observations:-

1. The LED successfully turns ON and OFF at 1-second intervals.
2. The GPIO pin responds correctly to Python commands, confirming proper configuration.
3. The **circuit works safely** without overheating or malfunctioning.

7.Conclusion:-

This experiment demonstrated the interfacing of an LED with Raspberry Pi 4. The **GPIO pins were successfully programmed** using Python to turn the LED ON and OFF. This experiment lays the foundation for more complex projects involving **sensors, motors, and displays** in embedded systems.

Expected Oral Questions:

1. What is the function of **GPIO pins** on Raspberry Pi?
2. Why do we use a **330Ω resistor** in the circuit?
3. How does the Python program control the LED?
4. What is the difference between **GPIO.BOARD** and **GPIO.BCM** modes?
5. What happens if we don't use **GPIO.cleanup()** in the program?

FAQs in Interview:

1. What are the different modes of GPIO pin configuration in Raspberry Pi?
2. How do you handle multiple LEDs using Raspberry Pi?
3. What is the role of RPi.GPIO library, and are there alternatives?
4. How can you modify the program to control the LED using a button?
5. What are some real-world applications of controlling LEDs with Raspberry Pi?